

CSEN 296B-2 | Week 4.2 Lab Submission

Complete UML Design Package for AstraNotes

Student Name: Alex Shirazi

Date: April 27, 2026

Project: AstraNotes

Chosen Technical Path: Python

Package Contents

- 1. Class Diagram
- 2. Object Diagram
- 3. Use Case Diagram
- 4. Activity Diagram
- 5. Deployment Diagram
- 6. Package Diagram
- 7. Written Rationale — how the views fit together
- 8. How AI Helped — what was used, refined, and rejected

Written Rationale

How the Six Views Fit Together

The six diagrams describe one consistent AstraNotes system from different perspectives. A class or relationship in one view appears consistently across all others.

Class → Object

Every object in the object diagram is a direct instance of a class in the class diagram. `secure1 : SecureNote` carries `encrypted_body` — an attribute that does not exist on plain `Note` — confirming the inheritance is architecturally meaningful, not decorative.

Class → Activity

The activity diagram's `is_private` branch creates a `SecureNote` instance and calls `.encrypt()` — both defined in the `SecureNote` class. The validation gate maps to `ValidationLayer.validate_note()` from the class diagram.

Use Case → Activity

The Create Secure Note use case with its «include» Encrypt Body is a drill-down of the `SecureNote` branch in the activity diagram. The diagrams agree: use case describes the goal, activity describes the flow.

Class → Package

Every class lives in exactly one package. `Note` and `SecureNote` in `astranotes.models`; `NoteService` in `astranotes.service`; `JsonFileRepository` in `astranotes.repository`; `PrivacyService` in `astranotes.privacy`.

Package → Deployment

Every component in the deployment diagram corresponds to a package. `SecureNote` and `PrivacyService` are grouped because `astranotes.privacy` is only consumed by `astranotes.models` — never directly by the service layer.

All Views → Requirements

`SecureNote` traces to FR-04 and SPR-01. It appears in every diagram: class (inheritance), object (`encrypted_body`), use case (Create Secure Note), activity (encrypt branch), deployment (grouped component), package (`astranotes.privacy`). The requirement is visible across all six views.

Applying the Professor's 3-Question Rule

- "Is it a kind of?" → `SecureNote` is a kind of `Note` → Inheritance ✓
- "Does it work with?" → `NoteService` works with `NoteRepository` → Dependency ✓
- "Does it strongly own?" → `NoteService` strongly owns `ValidationLayer` → Composition ✓
- "Does it work with?" → `SecureNote` works with `PrivacyService` → Dependency ✓

How AI Helped and What Was Refined

AI was used as a drafting partner across all six diagrams. The process followed the Working Agreement from Week 2.1: AI generates a first draft, I review against requirements and architecture, then accept, modify, or reject each suggestion.

What AI helped generate:

- Initial class structure — AI correctly identified Note, NoteRepository, NoteService, and PrivacyService as core classes from the requirement set.
- Use case list — AI generated all seven use cases from the feature list. The «include» relationships for encrypt/decrypt were suggested correctly and retained.
- Activity diagram flow — AI produced the correct sequence for Create Note including the validation gate and repository save path.
- Package structure — AI correctly separated cli, service, models, repository, and privacy into distinct packages.

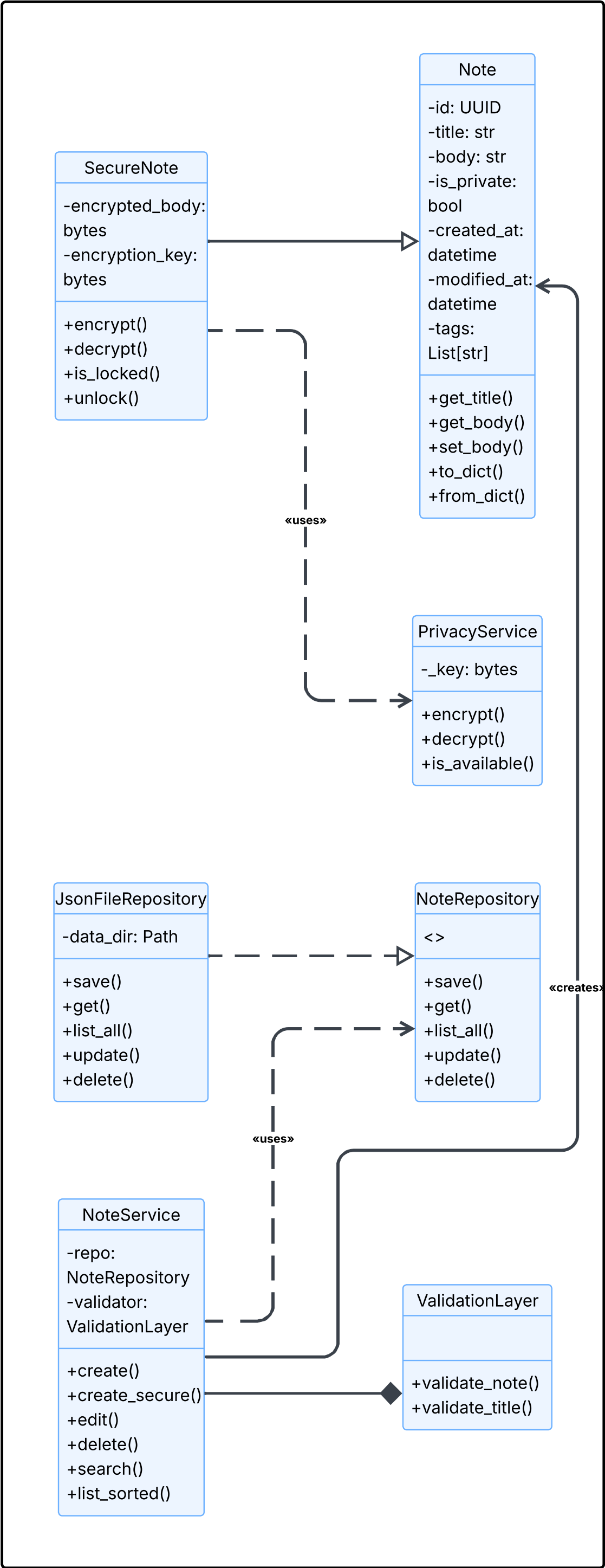
What was changed or rejected:

- AI initially handled privacy with only an `is_private` flag on Note — corrected to SecureNote as an explicit inheritance subclass, which is the professor's mandatory requirement.
- AI suggested UserSettings and NoteHistory classes — removed as not traceable to any current requirement. Cannot defend them from project artifacts.
- AI included a Cloud Backup node in the deployment diagram — removed since AstraNotes is local-only (confirmed in Week 3.2 governance review).
- AI placed PrivacyService inside the service package — moved to its own `astranotes.privacy` package to enforce clean downward-only dependency flow (NFR-02).
- AI's first object diagram used a diamond shape for NoteService — corrected to a plain rectangle. Object diagrams use rectangles only.

The final package reflects judgment applied on top of AI output — not AI output accepted blindly. Every class, relationship, and diagram type can be traced back to a specific requirement or architectural decision from earlier in the quarter.

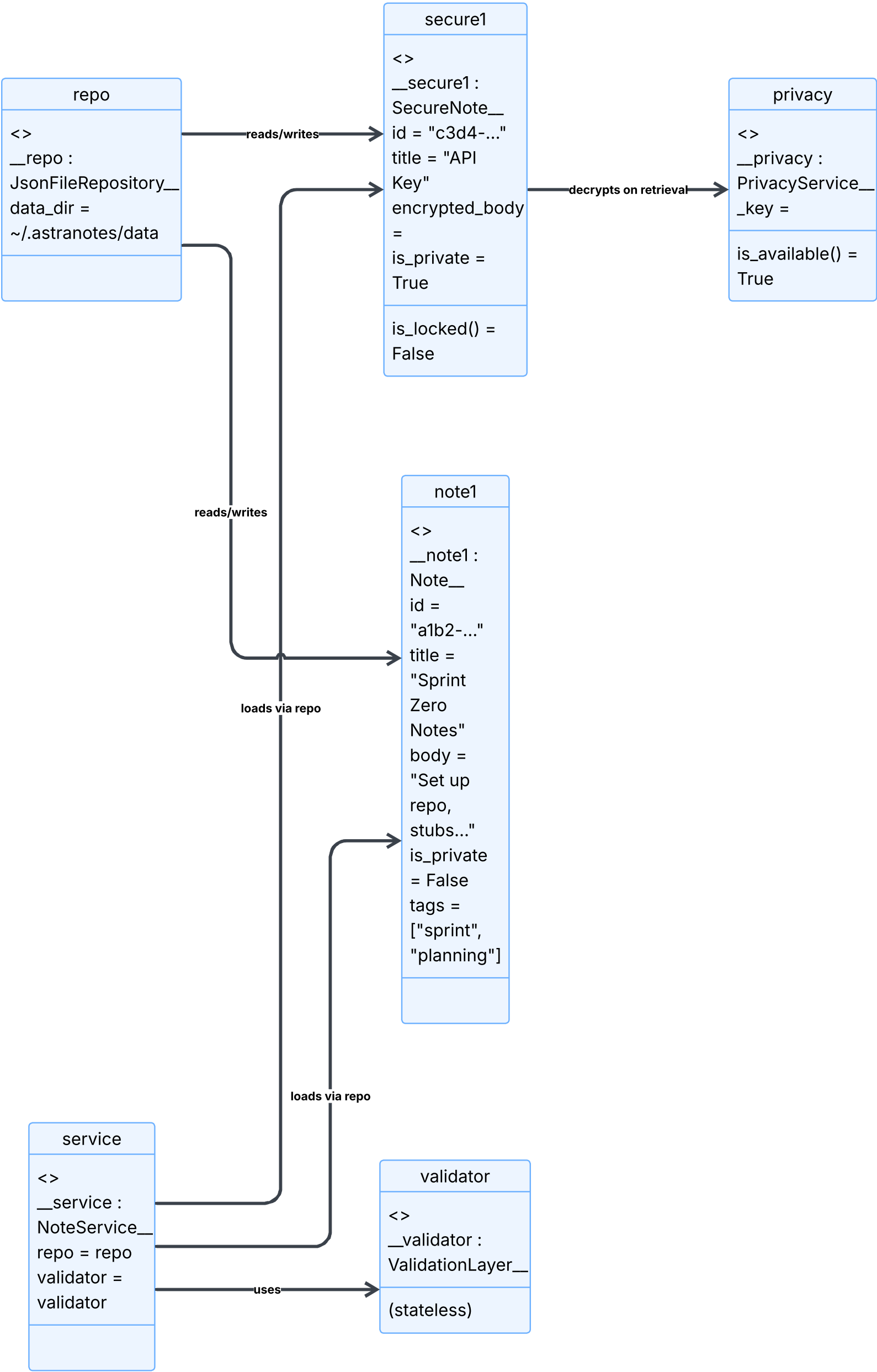
1. Class Diagram

Seven classes with UML visibility, attributes, and methods. SecureNote inherits from Note (open triangle). JsonFileRepository implements NoteRepository (dashed + open triangle). NoteService composes ValidationLayer (filled diamond) and depends on NoteRepository and PrivacyService.



2. Object Diagram

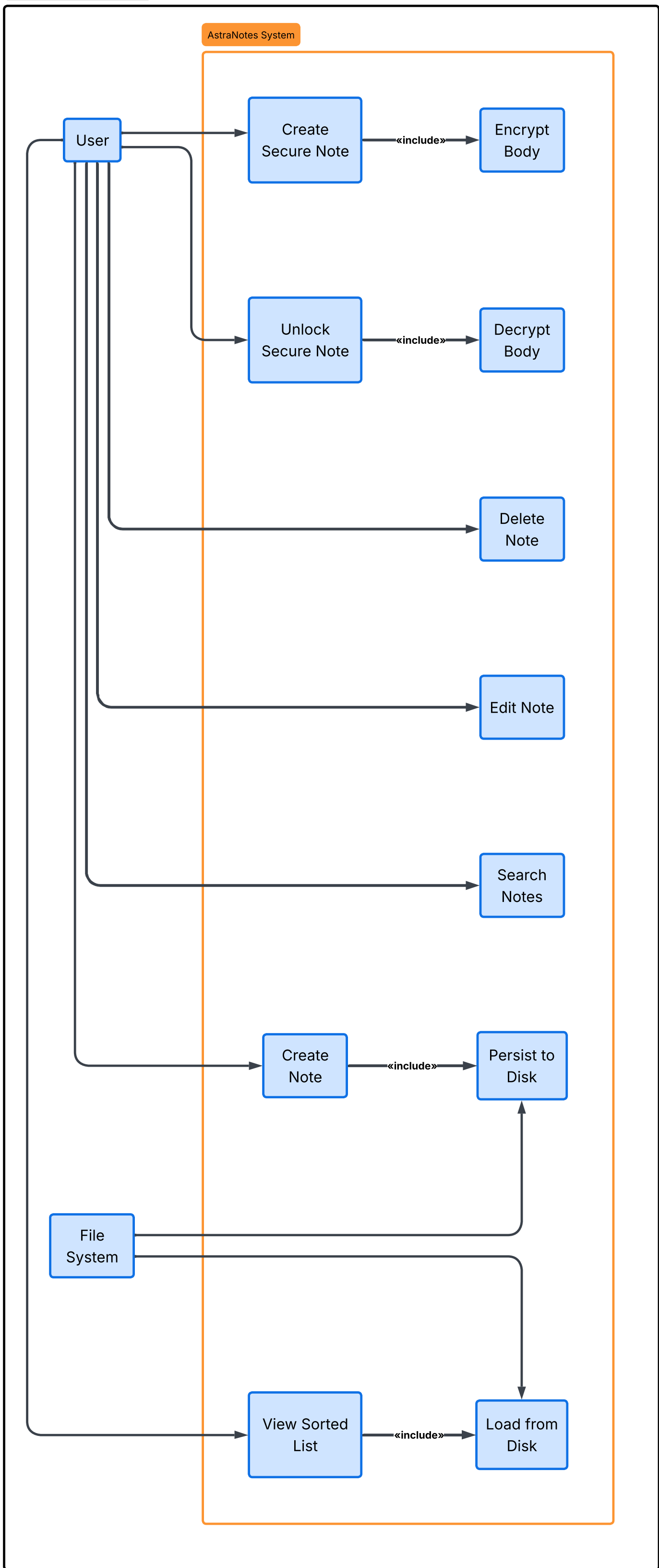
Runtime snapshot: note1 : Note (plaintext body) and secure1 : SecureNote (encrypted_body as Fernet bytes). Every object maps to a class from the class diagram and every attribute value matches its declared type.



3. Use Case Diagram

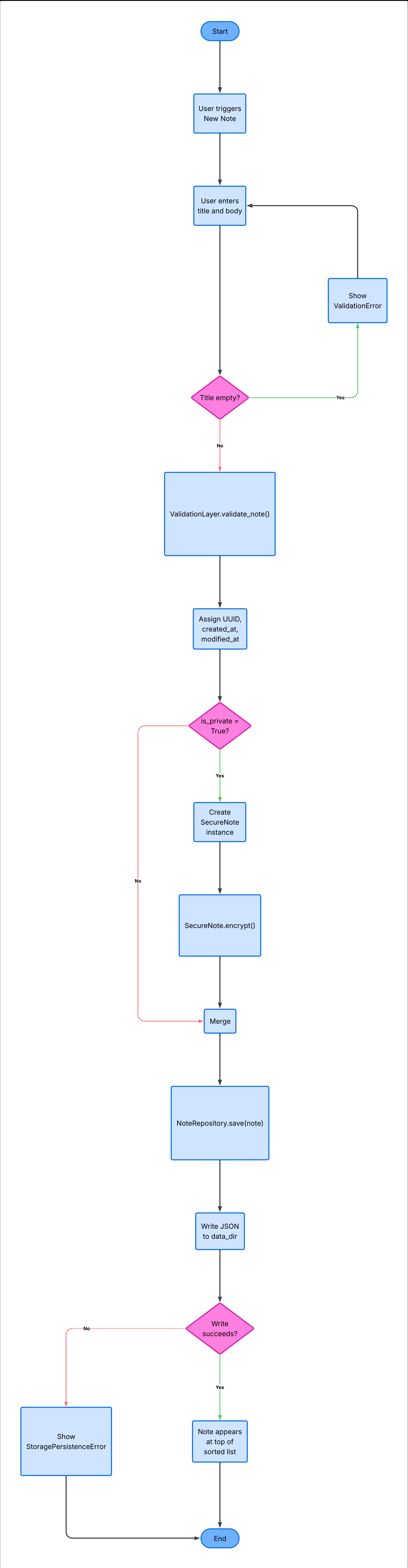
Seven use cases within the AstraNotes system boundary. User actor on left, File System actor on right. «include» relationships show Create Secure Note always invokes Encrypt Body, and Unlock Secure Note always invokes Decrypt Body.

AstraNotes Use Case Diagram



4. Activity Diagram

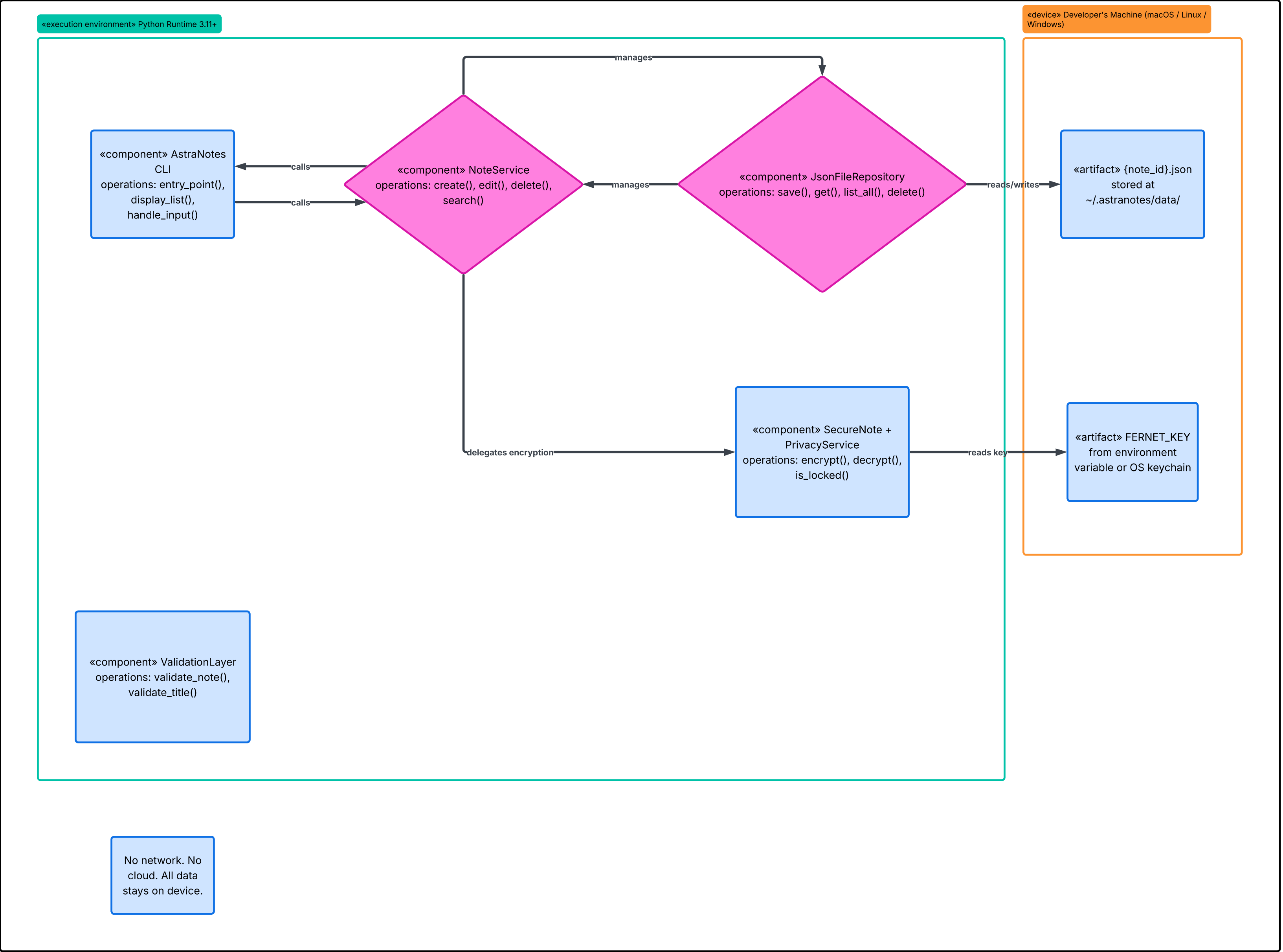
Create Note flow: validation gate (empty title raises `ValidationError`), `SecureNote` branch (Create `SecureNote` + encrypt when `is_private=True`), repository save, and storage failure path (`StoragePersistenceError`).



5. Deployment Diagram

Single-device local deployment. All components inside Python 3.11+ runtime. FERNET_KEY sourced from env var or OS keychain — never in source code or data directory (SPR-01). No network, no cloud.

AstraNotes Deployment Diagram



6. Package Diagram

Six Python packages with dependency arrows flowing downward only. astranotes.cli → service → models/repository/privacy. tests package depends on all five. No upward coupling (NFR-02).

AstraNotes Package Diagram

